

CSE 331

AUTOMATA AND COMPUTABILITY

PDA's

Summer 2026

Today's learning goals

Sipser Ch 1.4, 2.1

- Define push down automata
- Trace the computation of a push down automaton
- Design a push down automaton recognizing a given language
- Compare class of regular languages and class of CFLs

Birds' eye view

All languages over Σ

Context-free languages over Σ

Regular languages over Σ

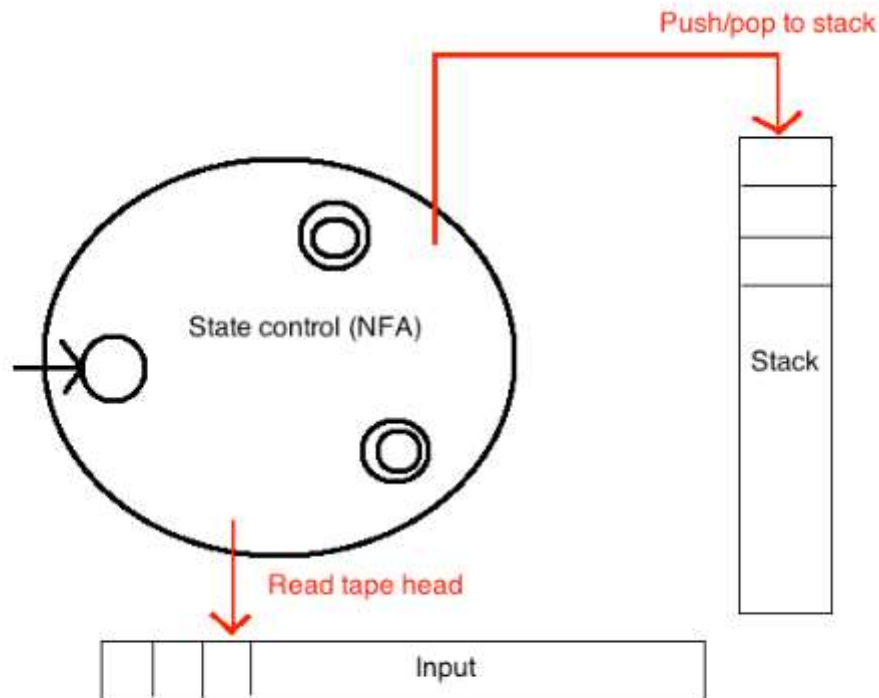
Finite languages over Σ



An alternative ...

Sipser p. 109

- NFA + stack



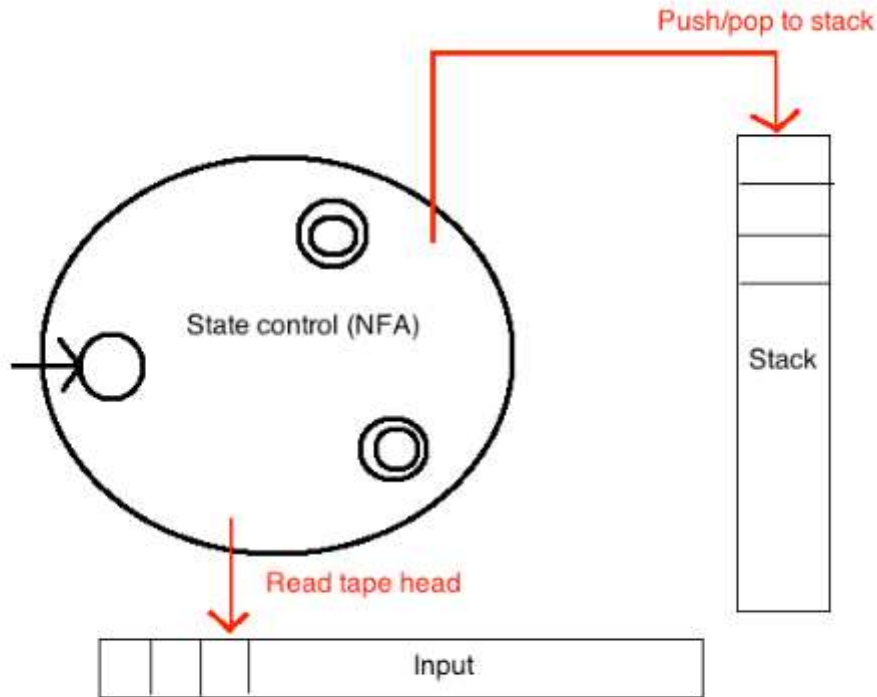
Pushdown automata

Sipser p. 109

- NFA + stack

At each step

1. **Transition** to new state based on current state, letter read, **and** top letter of stack.
2. (Possibly) **push or pop** a letter to (or from) top of stack



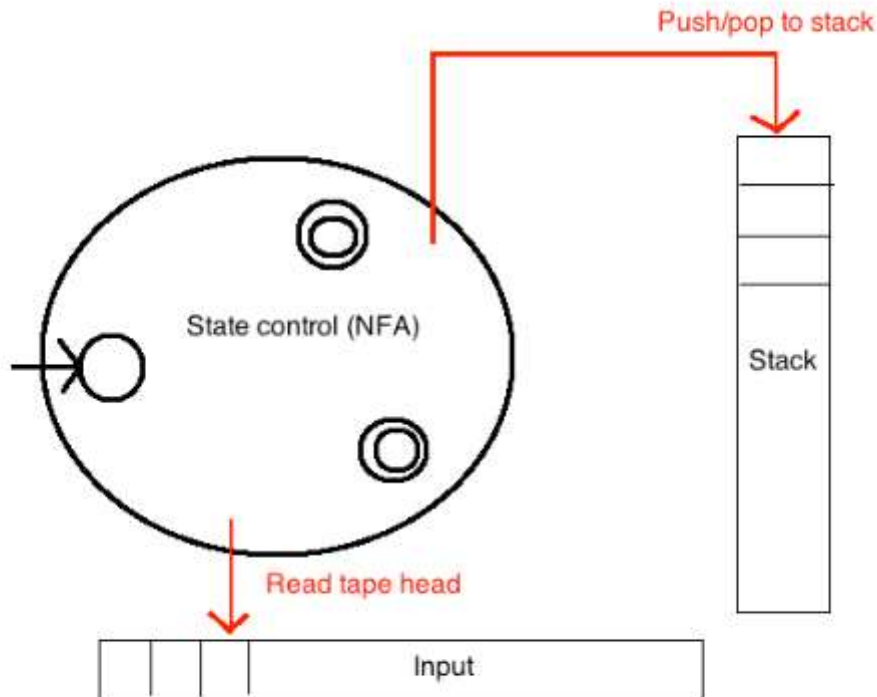
Pushdown automata

Sipser p. 109

- NFA + stack

Accept a string if

there is **some** sequence of states and **some** sequence of stack contents which processes the entire input string and ends in an accepting state.



State diagram for PDA

Sipser p. 109

If hand-drawn or in Sipser

State transition labelled $a, b \rightarrow c$ means

"when machine reads an a from the input and the top symbol of the stack is a b , it may replace the b with a c ."

State diagram for PDA

Sipser p. 109

If hand-drawn or in Sipser

State transition labelled $a, b \rightarrow c$ means

"when machine reads an a from the input and the top symbol of the stack is a b , it may replace the b with a c ."

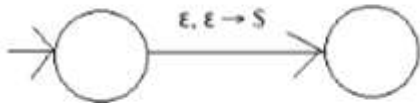
What edge label would indicate "Read a 0, pop 0 from stack, don't push anything to the stack"?

- A. $0, 0 \rightarrow \epsilon$
- B. $\epsilon, 0 \rightarrow 0$
- C. $0, \epsilon \rightarrow 0$
- D. $\epsilon \rightarrow \epsilon, 0$
- E. I don't know.

Useful trick

What would $\epsilon, \epsilon \rightarrow \$$ mean?

- A. Without reading any input or popping any symbol from stack, push \$
- B. If the input string and stack are both empty strings, push \$
- C. At the end of reading the input string, push \$ to top of stack
- D. I don't know.



Why is this useful?

Commonly used from initial state (at start of computation) to record top of stack with a special symbol)

"If the stack becomes empty and ..."

Formal definition of PDA *Sipser Def 2.13 p. 111*

A PDA is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$ where Q, Σ, Γ, F are all finite sets and

1. Q is the set of states
2. Σ is the input alphabet
3. Γ is the stack alphabet
4. $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$ is the transition function
5. $q_0 \in Q$ is the start state
6. $F \subseteq Q$ is the set of accept states.

Design a PDA

Build a PDA that accepts the language

$$\{ a^n b^n \mid n \in \mathbb{N} \}$$

Designing a PDA

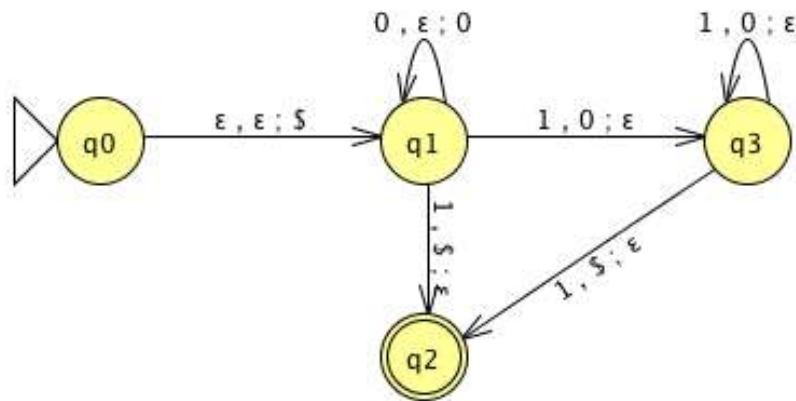
$$L = \{ 0^i 1^{i+1} \mid i \geq 0 \}$$

Informal description of PDA:

Read symbols from the input. As each 0 is read, push it onto the stack. As soon as 1s are seen, pop a 0 off the stack for each 1 read. If the stack becomes empty and there is exactly one 1 left to read, read that 1 and accept the input. If the stack becomes empty and there are either zero or more than one 1s left to read, or if the 1s are finished while the stack still contains 0s, or if any 0s appear in the input following 1s, reject the input.

Designing/Tracing a PDA

$$L = \{ 0^i 1^{i+1} \mid i \geq 0 \}$$



Sample computations?

- nondeterminism
- stack contents

Designing a PDA

$$L = \{ 0^i 1^{i+1} \mid i \geq 0 \}$$

Which of these CFGs generate L?

Assume V = set of variables mentioned in rules; $\Sigma = \{0, 1\}$, S start

- A. $S \rightarrow \varepsilon \mid 0S1$
- B. $S \rightarrow \varepsilon \mid 0S11$
- C. $S \rightarrow T1 \mid 0S1, T \rightarrow \varepsilon$
- D. $S \rightarrow \varepsilon \mid 0T1, T \rightarrow \varepsilon \mid 0T1$
- E. None of those

Design a PDA

Build a PDA that accepts the language
 $\{ w \mid w \text{ is a palindrome} \}$

Design a PDA

Build a PDA that accepts the language

$$\{ a^{3n}b^{2n} \mid n \in \mathbb{N} \}$$

Design a PDA

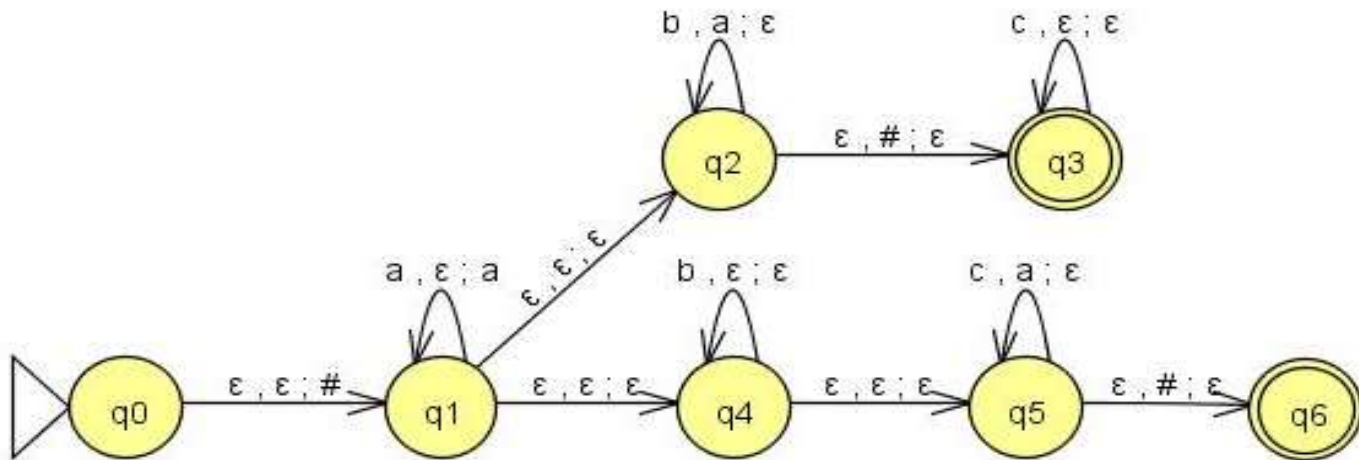
Build a PDA that accepts the language

$$\{a^n b^m \mid n > m \text{ and } m, n \in \mathbb{N}\}$$

Designing a PDA

$$L = \{ a^i b^j c^k \mid i=j \text{ or } i=k, \text{ with } i,j,k \geq 0 \}$$

Formal definition of PDA:



Challenge construction

- Given word w , and character c , define

$$\text{count}(w,c) = \text{count of } c \text{ in } w$$

- Consider the language

$$L = \{w_1 w_2 : w_1, w_2 \text{ in } \{0,1\}^*, \text{count}(w_1,0) = \text{count}(w_2,1)\}$$

- Design a CFG and PDA for it

Challenge PDA construction

$$L = \{w_1 w_2 : w_1, w_2 \text{ in } \{0,1\}^*, \text{count}(w_1, 0) = \text{count}(w_2, 1)\}$$

Challenge CFG construction

$$L = \{w_1 w_2 : w_1, w_2 \text{ in } \{0,1\}^*, \text{count}(w_1, 0) = \text{count}(w_2, 1)\}$$

Challenge: design a PDA

Build a PDA that accepts the language

$$\{x \in \{a, b\}^* : \#a \text{ in } x = \#b \text{ in } x\}$$

Design a PDA

Build a PDA that accepts the language

$\{w \mid \text{the number of a's in } w \text{ is not same as the number of b's in } w\}$

Design a PDA

Build a PDA that accepts the language

$\{x\#y \mid \text{the number of } 00 \text{ substring in } x \text{ is equal to the number of } 11 \text{ substring in } y\}$

Design a PDA

Build a PDA that accepts the language

$\{w \mid w \text{ starts and end with the same character}\}$

PDAs and CFGs are equivalently expressive

Theorem 2.20: A language is context-free if and only if some nondeterministic PDA recognizes it.

Consequences

- *Quick proof that every regular language is context free*
- *To prove closure of class of CFLs under a given operation, can choose two modes of proof (via CFGs or PDAs) depending on which is easier*

Closure properties of CFLs

$G_1 = (V_1, T, P_1, S_1)$ and $G_2 = (V_2, T, P_2, S_2)$

Assumption: $V_1 \cap V_2 = \emptyset$, that is, non-terminals are not shared

Theorem

CFLs are closed under union. L_1, L_2 CFLs implies $L_1 \cup L_2$ is a CFL.

Theorem

CFLs are closed under concatenation. L_1, L_2 CFLs implies $L_1 \bullet L_2$ is a CFL.

Theorem

CFLs are closed under Kleene star.

If L is a CFL $\implies L^*$ is a CFL.

More bad news: CFL not closed under intersection

Theorem

CFLs are *not* closed under intersection.

Theorem

CFLs are *not* closed under complement.

Example

$$L = \{ a^i b^j c^k \mid i=j \text{ or } i=k, \text{ with } i,j,k \geq 0 \}$$

To design a CFG that generates L ...

$$L = \{ a^i b^j c^k \mid i=j, \text{ with } i,j,k \geq 0 \} \cup \{ a^i b^j c^k \mid i=k, \text{ with } i,j,k \geq 0 \}$$

$$S \rightarrow Sc \mid T$$

$$T \rightarrow aTb \mid \varepsilon$$

Example

$$L = \{ a^i b^j c^k \mid i=j \text{ or } i=k, \text{ with } i,j,k \geq 0 \}$$

To design a CFG that generates L ...

$$L = \{ a^i b^j c^k \mid i=j, \text{ with } i,j,k \geq 0 \} \cup \{ a^i b^j c^k \mid i=k, \text{ with } i,j,k \geq 0 \}$$

$$S \rightarrow T \mid aSc$$

$$T \rightarrow Tb \mid \varepsilon$$

Designing a CFG – challenge!

Building a CFG to describe the language

$$\{ ww \mid w \in \Sigma^* \}$$

$$V = \{ S \}$$

$$\Sigma = \{ a, b \}$$

$$R = ?$$

S

This language is not context free!

This can be proved using a
“pumping lemma” for context free languages